

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track · Windows · Terminal-first

MONTH 3 — LAB GUIDE

Build a Data Detective Agent

An agent you point at any spreadsheet — it answers questions and draws charts.

Builds on Months 1–2. If you know variables, loops, lists, dictionaries and files, you're ready.

What you will build this month

- An agent that loads a real spreadsheet (CSV file) full of data.
- It answers questions like “how many rows?”, “what's the average?”, “which is biggest?”.
- It draws clear charts — bars, lines, histograms — from the data on request.
- It cleans messy data and works on any dataset you give it — shipped to GitHub as v1.0.

New this month: real data libraries

- This is your first month using **libraries** — pre-built toolkits other people wrote so you don't have to.
- **pandas** for handling tables of data, **matplotlib** for charts, and a little **numpy** for fast maths.
- You'll create one main file called **detective.py**, plus a sample data file to explore.

Month 3 at a glance

So far you've built everything by hand. This month you learn the professional way: standing on the shoulders of powerful libraries. Real data lives in spreadsheets, so your agent learns to read them, answer questions about them, and reveal what they contain through charts — exactly what a data analyst does.

Week	Focus	What you ship
Week 1	Install libraries & load real data	A program that loads a CSV and shows it (v0.1)
Week 2	Ask questions about the data	An agent that answers data questions in words (v0.2)
Week 3	Turn numbers into charts	An agent that draws charts on request (v0.3)
Week 4	Clean messy data & ship	A polished data detective on any dataset (v1.0)

A gentle heads-up: this month has a few more moving parts than the last two (installing libraries, a data file, charts opening in a window). Go slowly on Week 1 — once the tools are set up, the rest flows.

Week 1 — Install libraries & load real data

Goal of the week: set up the data libraries, get a sample spreadsheet, and load it into your program so you can see it.

Day 1 — Set up a fresh project & install libraries

Libraries don't come with Python — you install them once with a tool called pip. Today you set up the project and install the three you'll use all month.

Task 1.1 — Make the project folder

1. On your Desktop create a new folder called **detective**, then open it in VS Code (File → Open Folder).

Task 1.2 — Open the terminal inside VS Code

2. In VS Code's top menu, click **Terminal → New Terminal**. A terminal opens at the bottom, already pointed at your folder.

Task 1.3 — Install the libraries

3. In that terminal, type this and press Enter (it downloads the toolkits — give it a minute):

```
pip install pandas matplotlib numpy
```

✓ What you should see

Successfully installed pandas-... matplotlib-... numpy-...

If **pip** is not recognised, try **python -m pip install pandas matplotlib numpy** instead. This uses Python to run pip directly.

 **Checkpoint:** Your data libraries are installed. You only ever do this once per computer.

Day 2 — Get a sample spreadsheet

You need some data to explore. A CSV file is just a simple spreadsheet — plain text where commas separate the columns.

Task 2.1 — Create a sample CSV by hand

4. In VS Code, create a new file called **data.csv** and type this exactly (each line is a row; commas split the columns):

```
name,age,city,score
Alice,25,London,88
Bob,32,Paris,74
Charlie,28,London,91
Diana,45,Tokyo,67
Ethan,38,Paris,82
```

5. Save it. You've just made a spreadsheet with 5 people and 4 columns.

What is a CSV? “Comma-Separated Values.” The first row is the column names (headers). Every row after is one record. Excel and Google Sheets both export to this format.

 **Checkpoint:** You have a small, real dataset to work with all week.

Day 3 — Load the data with pandas

Now the exciting part — read that spreadsheet into your program in a single line, using pandas.

Task 3.1 — Read and show the data

6. Create `detective.py` and type:

```
import pandas as pd

data = pd.read_csv("data.csv")
print(data)
```

7. Save and run.

✓ What you should see

	name	age	city	score
0	Alice	25	London	88
1	Bob	32	Paris	74
2	Charlie	28	London	91
3	Diana	45	Tokyo	67
4	Ethan	38	Paris	82

`import pandas as pd` loads the library and nicknames it `pd`. `pd.read_csv` reads your file into a table pandas calls a **DataFrame** — think “a spreadsheet inside your program.”

File not found error? Make sure `data.csv` and `detective.py` are in the **same folder**, and that you opened that folder in VS Code.

🚩 **Checkpoint:** You loaded a real spreadsheet into your program and displayed it. This is the foundation of everything else.

Day 4 — Peek at the data

Real spreadsheets can have thousands of rows. pandas gives you quick ways to peek without printing everything.

Task 4.1 — First rows, size, and column names

8. Add these lines one at a time, running after each so you see what each does:

```
print(data.head(3))          # first 3 rows
print(data.shape)            # (rows, columns)
print(data.columns.tolist()) # list of column names
```

✓ What you should see

(the first 3 rows)

(5, 4)

['name', 'age', 'city', 'score']

`.head(3)` shows the top 3 rows, `.shape` tells you (rows, columns), and `.columns` lists the headers. These three are the first things every analyst checks.

🚩 **Checkpoint:** You can quickly inspect the size and shape of any dataset.

Day 5 — Look at one column

Often you care about just one column — all the scores, or all the ages. pandas makes that easy.

Task 5.1 — Select a single column

9. Add:

```
print(data["score"])      # every score
print(data["city"])       # every city
```

10. Run it and see just those columns.

Pick a column by its name in square brackets: `data["score"]`. This is just like looking up a value in a dictionary by its key — a skill from Month 2.

 **Checkpoint:** You can pull out any single column from the data by name.

Days 6–7 — Combine & ship v0.1

- Day 6: Meet a new common error — **KeyError** when you ask for a column name that doesn't exist (e.g. a typo like `data["Score"]` with a capital S). Fix by matching the exact header spelling from `.columns`.
- Day 7: Build v0.1 of `detective.py`: load the CSV, print a friendly welcome, show the shape, the column names, and the first few rows. Run it start to finish.

 **Checkpoint:**  Week 1 done. Your agent loads any spreadsheet and reports what's inside. Next week it answers questions.

Week 2 — Ask questions about the data

Goal of the week: turn your loader into an agent that answers real questions — counts, averages, maximums, and filters — in plain words.

Day 8 — Counting and averages

Task 8.1 — Basic maths on a column

11. pandas can do maths across a whole column instantly. Add:

```
print("How many people:", len(data))
print("Average score:", data["score"].mean())
print("Highest score:", data["score"].max())
print("Lowest score:", data["score"].min())
```

12. Save and run.

✓ **What you should see**

```
How many people: 5
Average score: 80.4
Highest score: 91
Lowest score: 67
```

`.mean()`, `.max()`, `.min()` work on any number column. pandas does the calculation across every row for you — no loop needed.

🚩 **Checkpoint:** You can compute counts, averages, highs, and lows on any number column.

Day 9 — Find the record behind the number

Knowing the highest score is 91 is good; knowing WHO got it is better.

Task 9.1 — Find the top row

13. Add:

```
top = data["score"].idxmax()      # position of the highest score
print("Top scorer:", data.loc[top, "name"])
```

✓ **What you should see**

```
Top scorer: Charlie
```

`.idxmax()` gives the row position of the biggest value; `.loc[row, column]` then reads a specific cell. Together they find the record behind the number.

🚩 **Checkpoint:** You can find not just the biggest value, but the row it belongs to.

Day 10 — Filtering: ask “which ones...?”

Filtering means keeping only the rows that match a condition — like “everyone who scored above 80.”

Task 10.1 — Filter rows by a condition

14. Add:

```
high = data[data["score"] > 80]
print(high)
print("How many scored above 80:", len(high))
```

✓ What you should see

(rows for Alice, Charlie, Ethan)
How many scored above 80: 3

Read the filter inside-out: `data["score"] > 80` marks each row true/false, and `data[ ... ]` keeps only the true ones. This is one of the most useful moves in all of data work.

🚩 **Checkpoint:** You can filter a dataset to just the rows that match a question.

Day 11 — Grouping: totals by category

Task 11.1 — Average score per city

15. Add:

```
print(data.groupby("city")["score"].mean())
```

✓ What you should see

city	
London	89.5
Paris	78.0
Tokyo	67.0

groupby bundles rows that share a value (all London rows together), then you do maths on each group. This answers questions like “average score per city” in one line.

🚩 **Checkpoint:** You can summarise data by category using groupby.

Days 12-14 — Build the question menu & ship v0.2

- Day 12: Build a menu loop (like Month 2's quiz loop) that shows options: 1) count, 2) average, 3) top scorer, 4) filter, 5) quit — and runs the matching code for the choice.
- Day 13: Make it feel like an agent — print answers in full friendly sentences, not raw numbers.
- Day 14: Ship v0.2 — an agent that answers several data questions through a menu. Test each option.

🚩 **Checkpoint:** End of Week 2: your agent answers real questions about the data in plain language.

Week 3 — Turn numbers into charts

Goal of the week: a picture beats a column of numbers. This week your agent draws charts with matplotlib — bars, lines, and histograms.

Day 15 — Your first chart

Task 15.1 — A simple bar chart

16. New file `chart_test.py`:

```
import matplotlib.pyplot as plt

names = ["Alice", "Bob", "Charlie"]
scores = [88, 74, 91]
plt.bar(names, scores)
plt.title("Scores")
plt.show()
```

17. Save and run. A window pops up showing a bar chart. Close the window to end the program.

✓ What you should see

A chart window opens
with three bars labelled Alice, Bob, Charlie

`import matplotlib.pyplot as plt` loads the chart tool as `plt`. `plt.bar(...)` makes bars, `plt.title(...)` adds a heading, and `plt.show()` opens the window.

The program pauses while the chart window is open — that's normal. Close the chart window and the program continues or ends.

🚩 **Checkpoint:** You can create and display a chart. A huge, satisfying milestone.

Day 16 — Chart straight from your data

Task 16.1 — Bar chart from the DataFrame

18. Back in `detective.py` (with pandas and matplotlib both imported at the top), add:

```
plt.bar(data["name"], data["score"])
plt.title("Score by person")
plt.xlabel("Name")
plt.ylabel("Score")
plt.show()
```

19. Save and run. Your real data becomes a labelled chart.

`xlabel` and `ylabel` name the bottom and side of the chart so a viewer understands it. Always label your charts.

🚩 **Checkpoint:** Your agent charts real data from the spreadsheet, properly labelled.

Day 17 — Different chart types

- Line chart with `plt.plot(...)` — good for trends over time.
- Histogram with `plt.hist(data["score"])` — shows how values are spread out.
- Task: try each on your data and notice which suits which question.

Day 18 — Save charts as image files

Task 18.1 — Save instead of (or as well as) showing

20. Before `plt.show()`, add:

```
plt.savefig("chart.png")
```

21. Run it, then look in your folder — a `chart.png` image has appeared that you can share.

 **Checkpoint:** Your agent can save charts as image files, not just display them.

Days 19-21 — Add charts to the menu & ship v0.3

- Day 19: Add chart options to your agent's menu (e.g. “6) show score chart”, “7) show city chart”).
- Day 20: Make each chart clear the previous one with `plt.clf()` so repeated charts don't overlap.
- Day 21: Commit and push to GitHub (same steps as before). This is v0.3.

 **Checkpoint:** Your data detective now answers questions AND visualises them. Live on GitHub.

Week 4 — Clean messy data & ship v1.0

Goal of the week: real data is messy — missing values, extra spaces, wrong types. This week your agent learns to clean it, work on ANY dataset, then ships as v1.0.

Day 22 — Find messy data

Task 22.1 — Spot missing values

22. Make a messy version `messy.csv` with a couple of blank cells (leave a value empty between commas), then load it and run:

```
data = pd.read_csv("messy.csv")
print(data.isnull().sum()) # counts blanks per column
```

✓ What you should see

```
name    0
age      1
score    2
```

`.isnull().sum()` counts the empty cells in each column — the first thing to check in any real dataset.

 **Checkpoint:** You can detect missing values in a dataset.

Day 23 — Fix messy data

- Fill blanks with a sensible value using `data["age"].fillna(0)`, or drop incomplete rows with `data.dropna()`.
- Remove stray spaces from text columns with `.str.strip()` (the `.strip()` you met in Month 2, applied to a whole column).

Day 24 — Work on ANY dataset

- Make your agent ask which CSV file to open at the start, instead of always using `data.csv`.
- Detect the column names automatically and let the user pick which column to analyse — so it works on datasets you've never seen.

Day 25 — Make it robust & friendly

- Handle bad input gracefully: unknown files, non-number columns, empty data — show a helpful message instead of crashing (try/except, gently).

Days 26-30 — Final polish & ship v1.0

- Day 26: Download a real CSV from the internet (e.g. a small public dataset) and point your agent at it. Fix anything that breaks on real data.
- Day 27: Write a `README.md` explaining what your detective does and how to run it.
- Day 28: Record a short demo — load a dataset, ask questions, show a chart.
- Day 29: Final commit and push (`git add .` → `git commit -m "v1.0"` → `git push`).
- Day 30: Show it off. You built an agent that analyses real spreadsheets and draws charts.



Month 3 complete

You now use professional libraries to load, question, visualise, and clean real data — the daily work of a data analyst. Just as importantly, you learned to stand on the shoulders of libraries instead of building everything by hand. Month 4 uses these data skills to give your next agent a sense of “similarity”, so it can recommend things — your first real taste of the maths behind AI.